

Lecture 6 - Case Study (Tư vấn dự án)

Mục lục

Lecture 6 (Case Study): Tư vấn giải pháp Security	2
Vì sao cần cụm Case Study?	2
Framework: 5 bước tiếp cận	2
Áp dụng vào 4 scenario	4
Một số nguyên tắc chung xuyên suốt	4
Ngoài STRIDE: các framework khác	5
Tóm tắt	5
Mini-quiz	5
6.2 Case Study: Web/SaaS Startup	7
Bối cảnh	7
Bước 1: Hiểu Context	7
Bước 2: Threat Modeling với STRIDE	7
Bước 3: Risk-prioritized Recommendation	8
Bước 4: Common Pitfalls	9
Tóm tắt	10
Bài học chính	10
6.3 Case Study: Fintech / Banking	11
Bối cảnh	11
Bước 1: Hiểu Context	11
Bước 2: Threat Modeling	11
Bước 3: Recommendation cho từng khía cạnh	12
Bước 4: Common Pitfalls fintech	14
Tóm tắt	15
Bài học chính	15
6.4 Case Study: IoT / Embedded	16
Bối cảnh	16
Bước 1: Hiểu Context	16
Bước 2: Threat Model cho IoT camera	16
Bước 3: Recommendation theo lớp	17
Bước 4: Patch model cũ	19
Bước 5: Common Pitfalls	19
Tóm tắt	20
Bài học chính	20
6.5 Case Study: Doanh nghiệp số hoá (Enterprise Cloud Migration)	21
Bối cảnh	21

Bước 1: Hiểu Context	21
Bước 2: Threat Landscape	21
Bước 3: Roadmap 12 tháng	22
Bước 4: Common Pitfalls	23
Bước 5: Metric để đo success	24
Tóm tắt	24
Bài học chính	25

Lecture 6 (Case Study): Tư vấn giải pháp Security

Tóm tắt một dòng: Cụm này khác bốn cụm trước về tinh thần: thay vì đi sâu vào một kỹ thuật, ta áp dụng tổng hợp mọi kỹ thuật đã học để **tư vấn** cho một dự án thực tế. Mỗi scenario có format: bối cảnh công ty, các vấn đề security cần quan tâm, recommendation cụ thể kèm trade-off.

Vì sao cần cụm Case Study?

Bốn cụm trước (Lec 1-5) dạy kỹ thuật: CIA, vulnerabilities, BMC, SMT, fuzzing. Đó là **what** và **how**: cái gì là bug, làm sao tìm, làm sao chứng minh.

Cụm này dạy **when** và **why**: trong một dự án cụ thể, ưu tiên gì, đầu tư vào đâu, đánh đổi như thế nào. Đây là kỹ năng của **security architect** hoặc **CISO**, không phải chỉ developer.

Khi bạn được giao một dự án mới, không có quy trình một-size-fit-all. Web app cần khác fintech cần khác IoT. Bài này giúp bạn:

1. Có một **framework chung** để tiếp cận mọi dự án.
2. Biết **các loại requirement** thường xuất hiện trong từng domain.
3. Biết **trade-off** giữa security với cost, performance, time-to-market.

Framework: 5 bước tiếp cận

Một quy trình chuẩn industry để tư vấn security:

Bước 1: Hiểu bối cảnh (Context)

Câu hỏi cần trả lời:

- Công ty làm gì? Sản phẩm chính là gì?
- Ai là user? Họ access từ đâu (mobile, desktop, IoT device)?
- Dữ liệu xử lý là loại nào (PII, payment, health, intellectual property)?
- Đối thủ và threat actor là ai (cybercrime, nation-state, competitor)?
- Budget và timeline có giới hạn gì?

Không có context, recommendation chỉ là generic. Ví dụ “dùng HTTPS” áp dụng mọi nơi, không phải tư vấn.

Bước 2: Threat Modeling

Threat modeling là quy trình formal để liệt kê threat. Có nhiều framework, phổ biến nhất là **STRIDE** của Microsoft:

Threat	Đe dọa	Vi phạm CIA
Spoofing	Giả mạo danh tính	Authenticity
Tampering	Sửa data trái phép	Integrity
Repudiation	Chối bỏ đã làm gì	Non-repudiation
Information Disclosure	Rò rỉ thông tin	Confidentiality
Denial of Service	Làm dịch vụ down	Availability
Elevation of Privilege	Leo thang quyền	Authorization

Cho mỗi component của hệ thống (web server, database, API gateway), enumerate STRIDE threats. Output: bảng threat liệt kê các attack vector tiềm năng.

Bước 3: Risk Assessment

Không phải threat nào cũng nguy hiểm như nhau. Risk = **Likelihood** × **Impact**:

Likelihood	Impact	Risk
High (mỗi tuần)	Critical (mất 100% data)	Catastrophic
Medium (mỗi tháng)	High (mất 10% data)	High
Low (mỗi năm)	Medium (mất 1% data)	Medium
Very low (mỗi 10 năm)	Low	Low

Sort threats theo risk, prioritize. Treat critical/high trước, low chấp nhận hoặc accept.

Tool industry: **DREAD** (Damage, Reproducibility, Exploitability, Affected users, Discoverability), **CVSS** (Common Vulnerability Scoring System).

Bước 4: Mitigation và Defense in Depth

Cho mỗi high-risk threat, define **mitigation**:

- **Preventive**: ngăn không cho threat xảy ra (vd: input validation chống injection).
- **Detective**: phát hiện khi threat xảy ra (vd: SIEM, IDS).
- **Corrective**: phục hồi sau khi threat xảy ra (vd: backup, incident response).
- **Deterrent**: làm threat ít khả thi (vd: log toàn diện cho audit).

Defense in depth: không dựa vào một lớp bảo vệ. Combine nhiều layer. Ví dụ chống SQL injection:

- Layer 1: parameterized query (preventive, mạnh nhất).
- Layer 2: web application firewall (preventive, secondary).
- Layer 3: least privilege database account (mitigate impact nếu SQLi thành công).
- Layer 4: monitoring database query lạ (detective).
- Layer 5: audit log + backup (corrective).

Nếu attacker bypass layer 1 (ví dụ bug trong ORM), layer 2-5 vẫn chặn được.

Bước 5: Trade-off và Roadmap

Security có cost. Mọi mitigation đều có:

- **Tiền**: tool license, infrastructure, headcount.

- **Performance:** encryption thêm latency, validation thêm CPU.
- **Usability:** 2FA tốn thời gian user, password complex khó nhớ.
- **Time-to-market:** review + audit chậm release.

Tư vấn tốt cân nhắc trade-off, không “max security everywhere”. Roadmap chia mitigation thành phases:

- **Phase 1 (must-have):** ngăn các bug critical (SQLi, auth bypass).
- **Phase 2 (should-have):** defense in depth (WAF, monitoring).
- **Phase 3 (nice-to-have):** advanced (BMC verification, formal proof).

Áp dụng vào 4 scenario

Cụm này có 4 bài tương ứng 4 domain:

Scenario	Đặc điểm	Bài học chính
Web/SaaS Startup	Move fast, MVP, scale fast	OWASP Top 10, auth, secrets management
Fintech/Banking	High regulation, money	PCI-DSS, key management, transaction integrity
IoT/Embedded	Constrained resource, physical access	Memory safety, secure boot, OTA, side-channel
Doanh nghiệp số hoá	Legacy + cloud, federation	Identity, network segmentation, BYOD, ransomware

Mỗi bài follow cấu trúc: 1. Bối cảnh và stakeholder. 2. Threat model (STRIDE applied). 3. Risk assessment. 4. Recommendation phase-by-phase. 5. Common pitfalls.

Một số nguyên tắc chung xuyên suốt

Trước khi đi vào từng scenario, đây là các nguyên tắc áp dụng cho mọi domain:

Principle 1: Least Privilege

Mỗi component chỉ có quyền tối thiểu cần thiết. Ví dụ: - Database account của app chỉ có SELECT/INSERT/UPDATE trên các bảng app dùng, không phải mọi bảng. - Container chạy với non-root user. - AWS IAM role chỉ có S3:GetObject trên bucket cụ thể, không phải * trên *.

Khi attacker compromise một component, blast radius bị hạn chế bởi privilege của component đó.

Principle 2: Zero Trust

Không tin tưởng dựa vào network location. “Inside corporate network” không có nghĩa “an toàn”. Verify mọi request, mọi connection.

Implementation: mutual TLS, service mesh (Istio), policy-based access (OPA).

Principle 3: Secure by Default

Mặc định setting là an toàn nhất. User phải explicit opt-in để giảm security.

Ví dụ: AWS S3 bucket mặc định private. User phải explicit set public. Trước 2018, mặc định là public, gây hàng nghìn data leak.

Principle 4: Defense in Depth

Không dựa vào một lớp. Multiple layers, mỗi layer có thể fail.

Principle 5: Fail Securely

Khi component fail, mặc định behavior là deny, không allow.

Ví dụ: nếu authorization service down, không cho phép action thay vì allow everything.

Principle 6: Privacy by Design

Không lưu dữ liệu không cần. Encrypt at rest và in transit. Implement data deletion theo GDPR/CCPA request.

Principle 7: Monitoring và Incident Response

Mọi component log đủ để truy ngược khi sự cố. Có playbook khi bug được phát hiện. Test playbook định kỳ.

Ngoài STRIDE: các framework khác

STRIDE là phổ biến nhất nhưng không phải duy nhất. Một số framework khác:

PASTA (Process for Attack Simulation and Threat Analysis): 7 stage, sâu hơn STRIDE.

ATT&CK (MITRE): catalog attack tactic và technique. Không phải framework threat modeling thuần, mà là **kho** để map mọi attack đã quan sát thực tế. Dùng cho threat intelligence.

Attack Tree: vẽ cây với root là goal attacker (ví dụ “đánh cắp customer data”), child là cách đạt goal (SQL injection, social engineering, insider). Trực quan, hay dùng trong workshop.

OCTAVE: focus organizational risk (people, process, technology). Phù hợp enterprise có nhiều stakeholder.

Trong tài liệu này dùng STRIDE làm primary framework. Bạn có thể combine với attack tree cho complex threat.

Tóm tắt

- Cụm Case Study về **tư vấn** dự án thực, không phải kỹ thuật cụ thể.
- **5 bước**: Context, Threat Model (STRIDE), Risk Assessment, Mitigation (defense in depth), Trade-off + Roadmap.
- **7 nguyên tắc chung**: Least Privilege, Zero Trust, Secure by Default, Defense in Depth, Fail Securely, Privacy by Design, Monitoring + IR.
- Framework khác: PASTA, ATT&CK, Attack Tree, OCTAVE.

Mini-quiz

Q1. Phân biệt threat modeling và risk assessment.

Threat modeling: liệt kê **mọi threat tiềm năng** mà attacker có thể thử. Không phán xét likelihood/impact. Output: comprehensive list.

Risk assessment: với mỗi threat, đánh giá **likelihood** (xác suất xảy ra) và **impact** (hậu quả). Risk = Likelihood × Impact. Output: prioritized list.

Threat modeling cho ta “biết những gì có thể xảy ra”. Risk assessment cho ta “biết phải lo cái nào trước”.

Ví dụ: threat “nation-state attack với 0-day exploit” có impact catastrophic nhưng likelihood low cho startup nhỏ. Risk medium-low. Trong khi đó “SQL injection vào form login” likelihood high (script kiddie), impact high (data leak). Risk critical.

Threat modeling là input cho risk assessment. Cả hai phải làm.

Q2. Cho ví dụ “defense in depth” trong context chống XSS.

Defense in depth cho XSS:

- **Layer 1:** Output encoding theo context (HTML, attribute, JS, URL). Mạnh nhất.
- **Layer 2:** Content-Security-Policy header: cấm inline script, eval, restrict origin. Block attack ngay cả khi layer 1 fail.
- **Layer 3:** HttpOnly + Secure + SameSite cookie. XSS không lấy được session cookie.
- **Layer 4:** Trusted Types API (modern browser): chỉ cho phép gán DOM từ “trusted” source. Block toàn bộ class DOM-based XSS.
- **Layer 5:** WAF (Web Application Firewall) chặn payload nghi ngờ ở edge.
- **Layer 6:** Monitoring và alert khi thấy <script> trong log request lạ.

Nếu attacker bypass output encoding (layer 1), CSP chặn. Nếu CSP có exception cho legitimate inline script, HttpOnly cookie bảo vệ session. Mỗi layer “cover” lỗ hổng của layer khác.

Triết lý: không lớp nào perfect. Combine multiple đảm bảo overall security.

Q3. Tại sao “Secure by Default” lại quan trọng hơn “Documentation tốt”?

User không đọc documentation. Đặc biệt: - Developer rush deadline. - Junior không biết cần đọc gì. - Sysadmin không expert security.

Setting mặc định ảnh hưởng **99%** user. Một mặc định không an toàn = 99% deployment không an toàn.

Ví dụ điển hình: - **MongoDB pre-3.0:** mặc định bind 0.0.0.0 (mọi network), không có auth. Hàng chục nghìn database leak online vì admin không biết phải secure. - **AWS S3 pre-2018:** mặc định ACL “public-read” cho nhiều operation. Hàng nghìn data leak. AWS phải đổi default + warning banner.

Documentation hữu ích nhưng không thay thế được mặc định an toàn. Best practice: mặc định mọi thứ deny/private, user explicit opt-in cho permission/exposure.

Trong Web framework: Django mặc định `DEBUG=False` cho production, CSRF protection enabled, XSS template escaping enabled. Không cần config thêm để “secure by default”.

Tiếp theo: **6.2 Case Study: Web/SaaS Startup**

6.2 Case Study: Web/SaaS Startup

Bối cảnh

Công ty A là startup phát triển sản phẩm SaaS cho thị trường B2B. Sản phẩm hiện tại là một web app cho phép user tải file Excel, app phân tích và sinh report.

Thông số: - 5 developer, 1 DevOps, không có security engineer. - Stack: React frontend, Node.js backend, PostgreSQL, deployed trên AWS. - 200 customer doanh nghiệp đang trả phí, tổng revenue \$50K/month. - Đang trong vòng gọi vốn Series A. - Customer yêu cầu báo cáo security cho audit của họ.

Đến gặp bạn để tư vấn: “Chúng tôi cần làm gì để security tốt, qua được customer audit, và sẵn sàng scale lên 10x customer trong 6 tháng?”

Bước 1: Hiểu Context

Đây là startup ở giai đoạn early-mid. Đặc điểm:

- **Time to market quan trọng:** không có budget cho security perfectionism.
- **Customer enterprise đòi hỏi:** SOC 2 audit, ISO 27001 ở chân trời.
- **Team nhỏ, không có dedicated security:** cần tool đơn giản, tự động.
- **Cloud-native:** AWS cung cấp nhiều primitive security sẵn.
- **Data sensitive:** file Excel của customer có thể chứa PII, financial data.

Recommend: focus vào “**good enough**” security cho 90% threat, không chạy theo bug edge case rare.

Bước 2: Threat Modeling với STRIDE

Liệt kê component chính:

1. **React frontend** (CloudFront + S3).
2. **API backend** (Node.js trên ECS).
3. **Database** (RDS PostgreSQL).
4. **File storage** (S3 bucket).
5. **Auth service** (Cognito hoặc Auth0).

Cho mỗi component, áp dụng STRIDE. Đây là output rút gọn cho **API backend**:

STRIDE	Threat cụ thể	Likelihood	Impact
Spoofing	Attacker gọi API với token giả	Medium	High
Tampering	SQL injection sửa data	High	Critical
Repudiation	User chối đã upload file XYZ	Low	Medium
Info Disclosure	API trả về data của user khác (IDOR)	High	High
DoS	Flood request, quá tải API	Medium	High
Elevation	User normal upgrade thành admin qua bug	Low	Critical

Top threats theo risk: SQLi, IDOR, DoS, spoofing token.

Bước 3: Risk-prioritized Recommendation

Phase 1 (Must-have, làm trong 1 tháng): Cover OWASP Top 10

OWASP Top 10 là list lỗ hổng web phổ biến nhất, update mỗi 3-4 năm. Phiên bản 2021:

1. **A01 Broken Access Control** (IDOR, auth bypass).
2. **A02 Cryptographic Failures** (weak crypto, store password plain).
3. **A03 Injection** (SQLi, XSS, command injection).
4. **A04 Insecure Design** (thiếu rate limit, abuse logic).
5. **A05 Security Misconfiguration** (default password, debug mode in prod).
6. **A06 Vulnerable Components** (lib có CVE).
7. **A07 Auth Failures** (weak password, session fixation).
8. **A08 Data Integrity Failures** (no signature on update).
9. **A09 Logging Failures** (no audit log).
10. **A10 SSRF** (server-side request forgery).

Cover từng cái:

A01 Access Control: - Mọi API endpoint check authorization explicit (không “if user logged in, allow”). - Object-level check: `if (resource.owner_id != current_user.id) return 403`. - Test với 2 user account, đảm bảo user A không thấy data user B.

A02 Cryptographic: - Password hash với bcrypt/argon2 (không MD5/SHA-1 thuần). - TLS 1.2+ everywhere, HSTS header. - Database encryption at rest (RDS encryption).

A03 Injection: - ORM với parameterized query (Prisma, TypeORM). - React auto-escape JSX (không dùng dangerouslySetInnerHTML). - Input validation library (Joi, Zod) ở mọi endpoint.

A04 Insecure Design: - Rate limit (express-rate-limit) ở endpoint nhạy cảm (login, password reset). - Business logic review: “có thể abuse được không?” (vd: trial expire bypass, coupon stack).

A05 Misconfiguration: - AWS Security Hub bật, follow recommendation. - S3 bucket không public, encryption at rest. - Container không chạy root.

A06 Vulnerable Components: - Dependabot bật trên GitHub, auto-PR khi có CVE. - `npm audit` trong CI, fail build nếu critical CVE.

A07 Auth Failures: - Dùng Auth0/Cognito thay vì self-built auth. - Password policy: min 8 chars, kết hợp letter + number, no common password. - Session timeout 30 phút idle, 24h absolute.

A08 Data Integrity: - HTTPS everywhere (đã có với CloudFront). - File upload: verify content-type, không tin tưởng extension.

A09 Logging: - CloudWatch log mọi authentication event, authorization deny, admin action. - Retention 1 năm minimum.

A10 SSRF: - Backend không gọi URL từ user input. Nếu cần, whitelist domain.

Effort estimate: 1 dev-month. Tool license: Auth0 ~\$200/month, Dependabot free, AWS Security Hub ~\$5/account/month.

Phase 2 (Should-have, làm trong 3 tháng): Defense in depth

WAF (Web Application Firewall): - AWS WAF với managed rule (Core Rule Set, SQL Injection Rule, XSS Rule). - Block payload SQLi/XSS biết trước, giảm load lên app. - \$5-10/month + rule cost.

Secrets Management: - Không hard-code secret trong code/env file. - AWS Secrets Manager hoặc HashiCorp Vault. - Rotate database password tự động hàng quý.

Container Image Scanning: - ECR có built-in scan (Trivy under the hood). - Block deploy nếu high/critical CVE.

SAST trong CI: - SonarQube hoặc Snyk Code. - Catch lỗi hổng tại commit time, không phải production.

Monitoring và Alerting: - CloudWatch alarm cho 5xx spike, login failure spike, slow query. - PagerDuty/Opsgenie cho on-call.

Effort estimate: 1 dev-month. Tool cost: \$100-300/month.

Phase 3 (Nice-to-have, làm trong 6 tháng): Audit-ready

SOC 2 Type 1 preparation: - Document policy (access control, incident response, data retention). - Hire auditor (3-6 tháng process, \$20-50K).

Penetration Test: - Hire 3rd party pen tester (HackerOne, Cobalt). - 2 weeks engagement, \$15-30K. - Fix findings critical/high, document.

Bug Bounty Program: - Public program trên HackerOne (sau khi pen test xong). - Pay \$500-5000 per bug tùy severity. - Incentivize community tìm bug 24/7.

Effort estimate: nhiều dev-month + budget \$50K+.

Bước 4: Common Pitfalls

Các sai lầm phổ biến của startup ở giai đoạn này:

Pitfall 1: Self-build authentication

Đừng tự viết auth. Dùng Auth0, Cognito, Firebase Auth, Clerk. Tự viết = bug. Auth library handle: - Password hashing đúng cách. - Session management. - Multi-factor auth. - OAuth integration. - Password reset flow.

Startup tự viết auth thường thiếu rate limit, OTP brute force protection, session fixation, ... Sửa bug auth là việc dài hơi.

Pitfall 2: Bỏ qua dependency CVE

npm install xong, dependency graph có hàng nghìn package. Mỗi tháng có CVE mới. Không scan định kỳ = thời điểm nào đó sẽ deploy code có CVE biết trước.

Bật Dependabot. PR auto. Merge sớm.

Pitfall 3: Secret trong code/.env commit

Easy mistake: tạo file .env.example với placeholder, dev copy thành .env và quên .gitignore. Push lên GitHub. Secret leak.

Solution: secret scanner trong pre-commit hook (TruffleHog, git-secrets). Và rotation policy: nếu nghi ngờ leak, rotate ngay.

Pitfall 4: Tin tưởng client-side validation

Frontend validate `if (price >= 0) submit()`. Attacker bypass: dùng curl, gửi negative price. Backend không check. Negative price stored, free product.

Rule: backend luôn validate, không tin client. Frontend validation là UX, không phải security.

Pitfall 5: Log nhạy cảm

`console.log(JSON.stringify(req.body))` log toàn body request, bao gồm password, credit card. Log file leak = breach.

Rule: structured logging với mask sensitive field. Tool: pino, winston với redaction.

Pitfall 6: Không có incident response plan

Khi breach xảy ra, panic. Không biết: - Ai notify ai (CTO, customer, regulator)? - Steps để contain breach. - Communication template cho customer. - Legal obligation (GDPR notification trong 72h).

Solution: viết incident response playbook trước. Drill mỗi quý.

Tóm tắt

- Phase 1 (1 tháng, ~\$200-500/month): cover OWASP Top 10.
- Phase 2 (3 tháng): defense in depth (WAF, secrets, scanning, monitoring).
- Phase 3 (6 tháng+): audit-ready (SOC 2, pen test, bug bounty).
- Tránh self-build auth, ignore CVE, secret leak, trust client.

Bài học chính

Web/SaaS startup cần balance “ship fast” và “secure”. Tool managed (Auth0, AWS WAF, Snyk) giảm rất nhiều effort. Roadmap chia phase, mỗi phase deliver value visible.

Khi customer enterprise audit, có document và evidence sẵn (logs, policy, pen test report) là chìa khoá pass.

Tiếp theo: 6.3 Case Study: Fintech/Banking

6.3 Case Study: Fintech / Banking

Bối cảnh

Công ty B là fintech startup phát triển ví điện tử cho thị trường Đông Nam Á. App cho phép user nạp tiền, chuyển tiền, thanh toán merchant, tiết kiệm.

Thông số: - 30 developer, 5 security engineer (đã có). - Stack: React Native mobile, Go backend, PostgreSQL + Redis, deploy on prem + AWS hybrid. - 5 triệu user, \$200M transaction/month. - License **Tổ chức cung ứng dịch vụ trung gian thanh toán** từ NHNN. - Phải tuân thủ **PCI-DSS** (vì xử lý thẻ tín dụng/debit).

Đến gặp bạn để tư vấn: “Chúng tôi sắp launch tính năng buy-now-pay-later, scale lên 20 triệu user. Cần thiết kế security cho tính năng mới và đảm bảo compliance audit pass.”

Bước 1: Hiểu Context

Fintech khác SaaS thông thường ở mức độ:

- **Regulation cực kỳ nặng:** NHNN, PCI-DSS, AML/KYC, GDPR (nếu user EU), local data residency law.
- **Money là target:** attacker có **financial incentive** rõ ràng. Nation-state, organized crime đầu tư resource.
- **Reputation critical:** một sự cố lớn = mất license, mất user.
- **Audit liên tục:** external auditor (Big 4), internal audit, regulator inspection.

Recommend: **không có shortcut**. Mọi feature phải qua security review. Budget phải để cho red team.

Bước 2: Threat Modeling

Bảng STRIDE cho component **payment processing**:

STRIDE	Threat	Mức quan trọng
Spoofing	Attacker giả user, transfer tiền	Critical
Tampering	MITM sửa amount giữa client và server	Critical
Tampering	Database breach, sửa balance	Critical
Repudiation	User chối đã transfer	High (legal)
Info Disclosure	Card number leak	Critical (PCI fine)
DoS	Attack ngày Black Friday	High
Elevation	Insider abuse, transfer ra account riêng	Critical

Thêm threat đặc thù fintech:

- **Account Takeover (ATO):** attacker steal credential, take over account.
- **Money mule:** account hợp pháp dùng để launder money.
- **Fraud network:** synthetic identity, ring of fake account.
- **API abuse:** thuê dịch vụ với credit card giả.

Bước 3: Recommendation cho từng khía cạnh

3.1 PCI-DSS Compliance

PCI-DSS là standard cho mọi entity xử lý payment card. 12 requirement chính:

Req	Mô tả ngắn
1	Firewall và network segmentation
2	Không default password
3	Protect stored cardholder data (PAN encryption)
4	Encrypt transmission (TLS)
5	Anti-malware
6	Secure development (SDLC, vuln scan)
7	Access control by business need
8	Identify and authenticate access
9	Physical access control
10	Track and monitor access
11	Regularly test (pen test, vuln scan)
12	Information security policy

Đặc biệt khó:

Req 3 (Encrypt PAN): - PAN (Primary Account Number) phải mask khi display (chỉ show 4 số cuối). - PAN phải encrypt at rest với key được manage trong HSM (Hardware Security Module). - KHÔNG được store CVV/CVC.

Req 10 (Logging): - Mọi access tới cardholder data phải log. - Log retention 1 năm minimum, 3 tháng online. - Log không thể bị sửa (write-once or signed).

Recommendation: dùng **PCI-DSS scope reduction**. Không store cardholder data trên server của bạn. Dùng **tokenization** từ payment processor (Stripe, Adyen, VNPAY). Token thay thế PAN, có scope hẹp hơn nhiều.

Scope reduction giảm compliance cost từ hàng trăm K USD/năm xuống hàng chục K.

3.2 Key Management

Trong fintech, key management là chỗ rất dễ sai và rất đắt khi sai.

Hierarchy of keys:

Master Key (HSM, never leaves)

↓ derives

KEK (Key Encryption Key)

↓ encrypts

DEK (Data Encryption Key)

↓ encrypts

Actual data (cardholder data, PII)

Best practice: - Master key trong HSM (Thales Luna, AWS KMS HSM-backed). Không bao giờ rời HSM. - Rotate DEK mỗi năm, re-encrypt data. - Rotate KEK mỗi 2-3 năm. - Có procedure recovery nếu HSM hỏng (multiple HSM cluster, key shares M-of-N).

Avoid: - Store key trong code, env var, config file. - Dùng cùng key cho mọi service (compromise 1 = compromise all). - Manual key handling (human typing key = mistake).

3.3 Transaction Integrity

Mỗi transaction phải bảo đảm:

Atomicity: A $\square$ B chuyển 100K. Hoặc cả A trừ + B cộng, hoặc không gì cả. Không có “A trừ nhưng B không nhận”.

Implementation: - Database transaction (PostgreSQL BEGIN ... COMMIT). - Distributed transaction nếu A và B khác database (2PC, saga pattern).

Idempotency: client retry transaction. Server thực hiện đúng 1 lần.

Implementation: - Client gửi idempotency_key UUID. - Server check key trước khi process. Nếu đã process, return previous result.

Signed audit log: - Mỗi transaction log với timestamp, amount, signed by HSM. - Audit log không thể sửa (append-only DB hoặc blockchain).

3.4 Anti-Fraud

Fraud detection là một ngành riêng. Cơ bản:

Rule-based: - Transaction > \$10K từ tài khoản mới $\square$ flag. - 10 transaction trong 1 phút từ cùng IP $\square$ block. - Transaction từ device chưa từng dùng $\square$ require 2FA.

ML-based: - Train model với historical fraud data. - Feature: amount, time, location, device fingerprint, behavior pattern. - Score mỗi transaction, > threshold $\square$ manual review.

Tool: Sift, Forter, hoặc build in-house.

Trade-off: false positive (legitimate user bị block) làm hại UX. False negative (fraud không catch) tổn tiền. Tune threshold theo cost.

3.5 KYC và AML

KYC (Know Your Customer): verify identity của user khi onboard. - Upload CCCD/CMND, selfie với CCCD. - Liveness check (không phải photo của photo). - Cross-check với government database nếu có.

AML (Anti-Money Laundering): - Monitor pattern transaction nghi ngờ (structuring, layering). - Report Suspicious Activity Report (SAR) tới regulator. - Hold transaction nghi ngờ pending review.

Tool: ComplyAdvantage, Refinitiv World-Check.

3.6 Insider Threat

Insider có quyền access database, có thể: - Trừ balance của user. - Tạo account giả, transfer tiền. - Export PII bán cho 3rd party.

Mitigation:

- **Least privilege:** developer không access production data. Read-only account cho debug.
- **4-eye principle:** action sensitive (refund > \$10K) cần 2 người approve.
- **Audit log everything:** mọi query database log với user ID, query text.

- **Behavior analytics:** nhân viên access data ngoài giờ làm hoặc volume bất thường → alert.

Insider threat khó vì attacker biết internal. Combine technical control (least privilege) + organizational (background check, separation of duties) + cultural (whistleblower policy).

Bước 4: Common Pitfalls fintech

Pitfall 1: Store CVV

CVV/CVC chỉ dùng để verify transaction tại điểm authorization. Sau đó **bắt buộc** xoá. Store CVV = vi phạm PCI-DSS, fine \$5K-100K/month.

Pitfall 2: Round error trong tính lãi/phí

```
fee = amount * 0.01
total = amount + fee
```

Với float, $100.001 * 0.01 = 1.000009999999999$. Tích lũy qua hàng triệu transaction tạo discrepancy. Cuối năm balance off bằng tỷ VND.

Fix: dùng **decimal** type (Python Decimal, Go big.Float, PostgreSQL NUMERIC). Không dùng float cho money.

Pitfall 3: Race condition trong balance update

```
def transfer(from_id, to_id, amount):
    balance = db.query(from_id).balance
    if balance >= amount:
        db.update(from_id, balance - amount)
        db.update(to_id, db.query(to_id).balance + amount)
```

Race: 2 transfer cùng lúc từ cùng account, cả 2 đọc balance trước update. Cả 2 pass check. Balance âm.

Fix: dùng SELECT FOR UPDATE hoặc database constraint CHECK (balance >= 0).

Pitfall 4: Timing attack trong compare token

```
if token == valid_token: # WRONG, không constant-time
    grant_access()
```

Compare normal stop tại byte khác đầu tiên. Attacker đo response time để guess byte by byte token.

Fix: dùng hmac.compare_digest() hoặc crypto.timingSafeEqual() (Node.js).

Pitfall 5: Webhook không verify signature

Payment processor (Stripe, VNPAY) gửi webhook khi transaction complete. App của bạn nhận, update balance.

Attacker phát hiện webhook endpoint, gửi fake webhook để credit balance miễn phí.

Fix: verify HMAC signature trong webhook header. Mỗi processor có document riêng.

Tóm tắt

- Fintech regulation rất nặng: PCI-DSS, NHNN, AML/KYC.
- **Tokenization** giảm PCI scope, đáng đầu tư.
- **Key management** với HSM, không bao giờ hard-code key.
- **Transaction integrity**: atomicity + idempotency + audit log.
- **Anti-fraud**: rule + ML, balance false pos/neg.
- **Insider threat**: least privilege, 4-eye, audit.
- Avoid: store CVV, float arithmetic, race condition, timing attack, unverified webhook.

Bài học chính

Fintech cần **defense in depth** ở mọi tầng vì stake cao. Không có “good enough”. Mỗi feature phải qua security review formal. Đầu tư vào red team + bug bounty là không tùy chọn.

Compliance không phải checkbox. Nó là continuous: auditor sẽ check lại mỗi năm, regulator có thể inspect bất kỳ lúc nào.

Tiếp theo: 6.4 Case Study: IoT/Embedded

6.4 Case Study: IoT / Embedded

Bối cảnh

Công ty C sản xuất camera an ninh thông minh cho gia đình. Camera có WiFi, microphone, motion sensor, kết nối cloud để stream video, nhận command từ mobile app.

Thông số: - Hardware: ARM Cortex-M4, 256KB RAM, 1MB flash, không có OS (bare metal C). - 500K camera đã bán, đang phát triển model mới. - Trước đó có **CVE** bị disclose: hardcoded admin password, attacker zombie camera vào botnet. - Đang gặp PR crisis, customer hỏi “có an toàn không”.

Đến gặp bạn để tư vấn: “Chúng tôi cần làm gì để model mới an toàn từ đầu, và làm sao patch model cũ?”

Bước 1: Hiểu Context

IoT khác web/mobile ở nhiều điểm quan trọng:

- **Resource constrained:** ít RAM, CPU yếu. Không thể chạy heavy security check.
- **Physical access:** attacker có thể mua thiết bị, mở xẻ, đọc flash. Không “trust the hardware”.
- **Long lifecycle:** thiết bị dùng 5-10 năm. Cần patch lâu dài.
- **Hard to update:** khác web (push một deploy), IoT cần OTA mechanism reliable.
- **Network heterogeneous:** WiFi, BLE, LoRa, Zigbee. Mỗi protocol có security riêng.

Recommend: **security from design**, không retrofit. Model cũ phải có **kill switch** (force update) nếu critical vuln.

Bước 2: Threat Model cho IoT camera

Attacker	Goal	Vector
Script kiddie	Botnet (Mirai-like)	Default password, exposed Telnet
Privacy invader	Spy on family	Steal video stream, eavesdrop
Burglar	Disable camera trước khi đột nhập	Jam WiFi, DoS camera
Nation-state	Surveillance	Implant backdoor in supply chain

STRIDE applied:

Threat	Risk
Spoofing camera (giả device ID, push fake video)	High
Tampering firmware	Critical
Repudiation (attack mà không log)	Medium
Info Disclosure (stream leak)	Critical
DoS (làm camera unresponsive)	High
Elevation (root via debug port)	Critical

Bước 3: Recommendation theo lớp

3.1 Memory Safety

ARM Cortex-M4 bare metal C: không có ASLR, không có DEP. Buffer overflow $\rightarrow$ arbitrary code execution dễ.

Khuyến nghị:

Switch ngôn ngữ nếu có thể: Rust biên dịch sang Cortex-M, memory safe by language. ESP32 đã có Rust toolchain.

Nếu phải dùng C: - Bật mọi compiler warning, treat as error. - Dùng MISRA-C subset (safety-critical C standard). - Static analyzer: PVS-Studio, Coverity, ESBMC. - Stack canary nếu CPU support (nhiều MCU không có). - Tránh strcpy, sprintf, gets. Dùng safe alternative.

Library: dùng tested library (Embedded TLS, mbedTLS) thay vì self-roll crypto.

3.2 Secure Boot

Vấn đề: attacker có physical access, có thể flash firmware giả. Boot từ firmware compromised, mọi security check trong firmware bypass.

Secure boot solution:

```
Boot ROM (hardware, immutable)
  ↓ verify signature of
Stage 1 bootloader
  ↓ verify signature of
Stage 2 bootloader (optional)
  ↓ verify signature of
Application firmware
  ↓ run
```

Mỗi stage verify next stage bằng signature (RSA hoặc ECDSA). Root of trust nằm trong Boot ROM (mạch điện, không thể flash lại).

Tool: ARM TrustZone, ESP32 Secure Boot, nRF52 với MCUboot.

Trade-off: cost design (cần CPU support), key management (private key signing không được leak), latency tăng (verify thêm boot time).

3.3 OTA Update

Critical vì IoT field deployed, không thể recall hàng triệu device.

OTA requirement:

1. **Signed firmware:** download xong verify signature trước khi apply.
2. **Atomic update:** nếu mất điện giữa chừng, vẫn boot được (A/B partition).
3. **Rollback:** nếu firmware mới crash 3 lần, auto rollback.
4. **Mandatory update:** critical CVE $\rightarrow$ force update, không cho user reject.
5. **Bandwidth-aware:** download tăng dần theo time, không peak.

Architecture điển hình:

Cloud: firmware repo + signing service

↓ HTTPS

Device: download firmware → verify signature → write to inactive partition

↓ reboot

Device: boot inactive partition → run → if healthy, commit; else, rollback

Tool: AWS IoT Device Management, Mender, MCUboot.

3.4 Authentication và Authorization

Pitfall điển hình IoT: hardcoded admin password (CVE đã bị attack).

Solution:

- **Unique device credential per unit:** mỗi camera có private key riêng, gen tại factory.
- **TLS mutual auth:** device và cloud authenticate lẫn nhau bằng certificate.
- **No default password:** lần đầu setup, user phải set password.
- **Account lockout:** 5 failed auth, lock 30 phút.

Token rotation: mobile app authenticate với cloud, lấy token để control camera. Token rotate hàng giờ, revoke khi user logout.

3.5 Network Security

Disable unused protocol: không có lý do để camera mở Telnet, FTP, UPnP. Disable hết.

WiFi credential storage: encrypt với device-unique key, không plain text trong flash.

Encrypted communication: - Camera ↔ Cloud: TLS 1.3. - Camera ↔ Mobile app (local network): DTLS hoặc end-to-end encryption.

Network segmentation guide: hướng dẫn user đặt IoT trên VLAN riêng, không cùng VLAN với laptop work.

3.6 Side-Channel Resistance

Side-channel attack: attacker đo physical signal (timing, power, EM) để extract secret.

Ví dụ: attacker đo time AES decryption. Time phụ thuộc key byte. Sau hàng nghìn measurement, extract key.

Mitigation: - Dùng AES implementation **constant-time** (intel AES-NI, ARM crypto extension). - Dùng masking: thêm random vào computation để hide signal. - Physical: enclosure hard to open, tamper-evident.

Cho consumer IoT, side-channel thường low priority (chi phí attack cao, không scale). Nhưng cho thiết bị banking, military, side-channel quan trọng.

3.7 Privacy

Camera xử lý video, audio: rất nhạy cảm.

Best practice:

- **Local processing:** motion detection, face recognition chạy on-device, không send everything cloud.

- **End-to-end encryption:** video encrypt từ camera, chỉ user mobile app decrypt. Cloud không xem được.
- **Data minimization:** chỉ lưu metadata cần thiết, không lưu video không user explicit.
- **Transparent privacy policy:** rõ ràng “chúng tôi không truy cập video của bạn”.
- **GDPR / privacy law compliance:** user có quyền delete data, export data.

Bước 4: Patch model cũ

Đối với 500K camera đã bán có CVE:

Step 1: Disclosure

- Public security advisory chi tiết CVE, scope ảnh hưởng.
- Coordinate với CERT (CISA, VNCERT) nếu critical.

Step 2: Patch development

- Fix code, sign firmware.
- Test extensive ở dev lab + bên ta.

Step 3: OTA push

- Push update tới mọi device.
- Force update sau 30 ngày nếu user không apply.
- Monitor success rate.

Step 4: Devices không update được

- Một số device không lên mạng > 30 ngày: không nhận được patch.
- Có thể disable một số tính năng remote control nếu firmware quá cũ.
- Worst case: recall thiết bị (đắt nhưng cần thiết cho critical vuln).

Step 5: Process improvement

- Why hardcoded password slipped? □ audit code review process.
- Implement Static Analyzer cho mọi commit.
- Mandatory security training cho developer.

Bước 5: Common Pitfalls

Pitfall 1: Hardcoded credential

Vẫn xảy ra! Developer hard-code cho debug, quên remove. Test với grep + signature scanner trong CI.

Pitfall 2: Backdoor “support”

Backdoor cho support team troubleshoot. Backdoor leak □ attacker dùng.

Solution: không có backdoor. Support cần access □ user explicit consent + temporary credential.

Pitfall 3: Đặt private key trong flash

Private key cho signing là tài sản quý. Nếu attacker đọc flash, lấy được key, có thể sign firmware giả.

Solution: private key trong **secure element** (Microchip ATECC608A, NXP A71CH). Hardware-protected, key never exits.

Pitfall 4: Update mechanism không tested

OTA update viết, không test edge case (mất điện, network drop giữa chừng). Khi push thật, brick hàng nghìn device.

Solution: chaos engineering. Simulate fail mid-update. A/B partition + rollback.

Pitfall 5: Telemetry leak privacy

Telemetry để debug: location, device ID, behavior. Leak qua cloud breach.

Solution: anonymize, aggregate. Document rõ telemetry collect cái gì.

Tóm tắt

- IoT có constraint riêng: resource, physical, lifecycle.
- **Memory safety**: ưu tiên Rust hoặc C với heavy linting.
- **Secure Boot**: chain of trust từ Boot ROM.
- **OTA**: signed, atomic, rollback, mandatory critical.
- **Unique credential per device**: không hardcoded.
- **Network**: disable unused protocol, segment VLAN.
- **Privacy**: local processing, E2E encryption, data minimization.

Bài học chính

IoT security khó nhất ở **physical access** và **lifecycle**. Web app có thể patch trong giờ; IoT mất tuần/tháng và một số device không bao giờ patch được.

Vì thế: **design for security from start**, plan OTA mechanism từ ngày 1, expect attacker có physical access từ ngày 1. Retrofit security cực kỳ đắt và không hiệu quả.

Tiếp theo: 6.5 Case Study: Doanh nghiệp số hoá

6.5 Case Study: Doanh nghiệp số hoá (Enterprise Cloud Migration)

Bối cảnh

Công ty D là tập đoàn sản xuất truyền thống (đồ gia dụng), 5000 nhân viên, đang trong quá trình “digital transformation”:

- Trước: ERP on-prem (SAP), mọi nhân viên access qua VPN.
- Đang: chuyển ERP lên SaaS (Workday, Salesforce), Office 365, deploy app web nội bộ trên AWS.
- Đang: cho phép BYOD (Bring Your Own Device), nhân viên làm việc từ xa.
- Trước: chỉ có IT support, không security team riêng.
- Mới: tuyển CISO + 5 security engineer.

Đến gặp bạn để tư vấn (bạn là consultant cho CISO mới): “Chúng tôi cần roadmap security 12 tháng để hỗ trợ digital transformation an toàn.”

Bước 1: Hiểu Context

Enterprise migrate to cloud có challenge riêng:

- **Legacy system phức tạp:** ERP cũ chứa logic 20 năm, không thể rewrite. Phải integrate với cloud.
- **Identity sprawl:** nhân viên có account ở 50+ system. Khó manage.
- **Compliance overlay:** SOX (financial), GDPR (PII), industry-specific (FDA, NIST).
- **People process change:** nhân viên không quen tool mới. Training cost lớn.
- **Vendor lock-in concern:** dùng nhiều SaaS, khó migrate sau.
- **Ransomware target:** enterprise = wallet to attacker.

Recommend: **roadmap chia phase**, đo bằng business outcome chứ không phải tool count.

Bước 2: Threat Landscape

Enterprise face threat khác startup:

Threat	Mức độ	Loss potential
Ransomware	Critical	rất cao (millions USD)
Insider data theft (M and A info, IP)	High	cao
Phishing → compromise admin	Critical	rất cao
Cloud misconfiguration → data leak	High	cao
Third-party breach (vendor)	High	cao
Compliance fine	Medium	trung bình

Top attacker tactic:

1. **Phishing:** email spoof, target nhân viên có quyền cao.
2. **Credential stuffing:** dùng credential leak từ breach khác.
3. **Supply chain:** compromise vendor có access vào hệ thống.
4. **Insider:** nhân viên bất mãn hoặc bị mua chuộc.
5. **Ransomware:** encrypt data, đòi tiền chuộc.

Bước 3: Roadmap 12 tháng

Tháng 1-3: Foundation

Identity và Access Management (IAM)

Vấn đề lớn nhất ở enterprise migrate: identity ở mọi nơi. AD on-prem, Salesforce, Workday, AWS, ...
Mỗi system account riêng.

Solution: Identity Federation với Single Sign-On (SSO).

Employee → Identity Provider (Azure AD / Okta) → 50+ SaaS / AWS

Một login cho mọi thứ. Khi nhân viên nghỉ việc, disable trong IdP, mọi system mất access.

Implementation: - Choose IdP: Azure AD (nếu đã có Microsoft) hoặc Okta (multi-cloud friendly). - Federate SAML / OIDC với mọi SaaS. - Implement SCIM cho auto-provisioning.

Multi-Factor Authentication (MFA): - Bắt buộc cho mọi access. - Push notification (Duo, Microsoft Authenticator) > SMS. - FIDO2 hardware key cho admin (cao nhất).

Privileged Access Management (PAM): - Admin account dùng riêng (không phải daily account). - Just-in-time access: request quyền admin khi cần, expire sau X giờ. - Tool: CyberArk, BeyondTrust.

Tháng 3-6: Network và Endpoint

Zero Trust Network:

Cũ: VPN cho mọi access. Sai vì: - VPN compromise = full network access. - Lateral movement easy. - Không scale với BYOD.

Mới: **Zero Trust**. Mỗi request verify regardless network location.

Implementation: - **Software-defined perimeter** (Zscaler, Cloudflare Access). - **Microsegmentation**: app A không thấy app B unless explicitly allowed. - **Service mesh** (Istio) cho microservice.

Endpoint Detection and Response (EDR):

Cài agent trên mọi laptop, server. Agent monitor: - Process execution suspicious. - File access pattern. - Network connection. - Behavior anomaly.

Tool: CrowdStrike, SentinelOne, Microsoft Defender for Endpoint.

Mobile Device Management (MDM):

BYOD = phone cá nhân, không control hoàn toàn.

Solution: MDM enforce policy: - Encrypt device. - Password 6 digit minimum. - Remote wipe nếu mất. - Containerize work data (Microsoft Intune, MobileIron).

Tháng 6-9: Data Protection

Data Loss Prevention (DLP):

Monitor data leaving company: - Email với attachment có PII □ block hoặc encrypt. - Upload tới personal cloud (Dropbox cá nhân) □ block. - Print large data set □ log + approval.

Tool: Microsoft Purview, Symantec DLP.

Cloud Access Security Broker (CASB):

Visibility và control trên SaaS usage: - Discover Shadow IT (employee dùng SaaS không qua IT). - Enforce policy (only allowed SaaS). - DLP for SaaS data.

Tool: Netskope, Zscaler, Microsoft Defender for Cloud Apps.

Backup và Ransomware:

Ransomware kéo critical attack vector cho enterprise.

Mitigation: - **3-2-1 backup**: 3 copy, 2 medium khác, 1 offsite. - **Immutable backup**: không thể delete trong 30 ngày (S3 Object Lock, Veeam Immutable). - **Air-gapped backup**: 1 copy hoàn toàn offline. - **Test restore**: monthly drill, không chỉ test backup work, test recovery time. - **Endpoint protection** với behavioral detection (CrowdStrike, SentinelOne). - **Email security** (phishing là ingress chính): Proofpoint, Mimecast.

Tháng 9-12: Compliance và Governance

Information Security Policy:

Document policy formal cho: - Acceptable Use. - Data Classification (Public, Internal, Confidential, Restricted). - Incident Response. - Vendor Risk Management. - BCP/DR (Business Continuity / Disaster Recovery).

Security Awareness Training:

Mandatory cho mọi nhân viên: - Phishing simulation hàng tháng. - Annual security training. - Onboarding training cho new hire.

Tool: KnowBe4, Cofense.

Vendor Risk Management:

Mọi vendor có access vào data company phải: - Pass security questionnaire. - Có SOC 2 report (hoặc tương đương). - Sign DPA (Data Processing Agreement). - Review hàng năm.

Tool: OneTrust, ProcessUnity.

Compliance Certifications:

Tùy industry: - **SOC 2 Type 2**: 6 tháng observation period, \$50-100K cost. - **ISO 27001**: international standard, \$30-80K. - **GDPR**: nếu có EU user, designate DPO. - **HIPAA**: nếu xử lý health data.

Bước 4: Common Pitfalls

Pitfall 1: “Lift and shift” không secure

Move workload từ on-prem lên cloud nguyên xi. Security model on-prem không apply cho cloud: - Firewall rule on-prem khác AWS Security Group. - IAM concept on-prem khác cloud.

Solution: re-architect cho cloud-native security (security group, IAM role, KMS, ...).

Pitfall 2: Quá nhiều tool, overlap

Mua mọi tool quảng cáo: SIEM, SOAR, XDR, CSPM, DLP, ... Team không dùng được hết. Lãng phí budget.

Solution: chọn 1 tool / category, integrate well. Less is more.

Pitfall 3: Security as blocker, không enabler

Security team say “no” mọi thứ. Developer bypass, dùng shadow IT.

Solution: security partnership. Đề xuất alternative thay vì reject. “Bạn muốn dùng tool X, tốt cho UX, nhưng concern A, B. Hãy cùng giải quyết.”

Pitfall 4: Bỏ qua phishing training

Bỏ tiền vào tool nhưng không train nhân viên. 95% breach bắt đầu bằng phishing. Training cheap, ROI cao nhất.

Pitfall 5: Không test incident response

Có IR playbook nhưng không drill. Khi real incident, panic, slow.

Solution: tabletop exercise mỗi quý. Simulate breach scenario, walk through response.

Pitfall 6: Bỏ qua Insider Threat

Focus external attacker, quên insider. Disgruntled employee, departing employee. 30% breach involve insider.

Solution: User Behavior Analytics (UBA), monitoring privileged access, exit interview với data check.

Bước 5: Metric để đo success

12 tháng sau, đo:

Metric	Target
% employee with MFA	100%
% critical asset có EDR	100%
Phishing click rate	< 5% (after training)
Mean Time to Detect (MTTD)	< 1 hour
Mean Time to Respond (MTTR)	< 4 hours
% vendors with SOC 2	80%+
Backup restore test success	100% monthly
Open critical vulnerabilities > 30 days	0

Không có “100% secure” KPI. Mục tiêu là **reduce risk** measurably.

Tóm tắt

- Enterprise migrate cloud cần roadmap 12+ tháng.
- **Tháng 1-3**: Identity (SSO, MFA, PAM).
- **Tháng 3-6**: Network (Zero Trust) + Endpoint (EDR, MDM).
- **Tháng 6-9**: Data (DLP, CASB, Backup).
- **Tháng 9-12**: Governance (Policy, Training, Vendor risk, Compliance).
- Avoid: lift-and-shift, tool overlap, security as blocker, no IR drill, insider blind spot.

Bài học chính

Enterprise security là về **process và people**, không chỉ technology. Nhân viên là target chính (phishing). Vendor là attack vector growing (supply chain). Insider luôn risk.

Tool quan trọng nhưng không đủ. CISO thành công là người **partner với business**, không “police”. Security enable business chuyển đổi, không block.

Kết thúc Cụm 5 (Case Study). Bạn đã có 4 framework để tư vấn 4 domain phổ biến: Web/SaaS, Fintech, IoT, Enterprise. Combine với kỹ thuật đã học ở Lec 1-5, bạn đủ kiến thức để approach hầu hết security project trong industry.